WikiApp

Code Walkthrough & Viva Preparation Report

Part A & Part B

BIT235 - Object Oriented Programming
Assessment 2
Semester 1, 2026

Author: Movindu Lochana
Student ID: s1577380
GitHub: https://github.com/DustyxT/WikiApp

A beginner-friendly explanation of every code block in the project, designed to help me understand and explain my own work during the viva.

Contents

Overview

Part A - Login (Files)

Part B - Database, Wiki, CRUD (Files)

24. admin/form.html	Create / edit article form
25. admin/confirm-delete.html	Delete confirmation page

Reference & Viva

26. Annotation Cheat Sheet	Every annotation in plain English
27. Thymeleaf Cheat Sheet	Template syntax explained
28. Likely Viva Questions	30+ Q&A covering both parts
29. One-Minute Elevator Pitch	Memorise this
30. Vocabulary Glossary	Key terms defined

1. Project Overview

WikiApp is a Spring Boot web application built in two stages.

Part A implemented the Wiki administrator login screen with hard-coded credentials. The user submits a form, the controller hands the data to a service, the service validates it, and the user lands on a welcome or error page.

Part B extended the application with a real H2 database, full CRUD operations, a public wiki interface, and an admin back-end. The hard-coded credentials were replaced with a JPA-managed Admin entity, and articles are now stored in the database. The Controller layer was extended with HttpSession-based authentication so that admin pages can only be accessed when logged in.

What both parts share

The MVC pattern (Model - View - Controller) and a three-layer architecture (Controller -> Service -> Repository). Each class has exactly one job. When Part B added a database, the Controller and Service did not need to be rewritten - only the Repository changed (from a hard-coded class to a JPA interface). That is the reward of a well-layered design.

HD-rubric criteria targeted

MVC Structure	Three packages: controller / service / repository, plus model
Controllers / Mappings	@GetMapping and @PostMapping; full request flow works for both parts
Form & Data Flow	@ModelAttribute on LoginForm and Article; Model carries data to views
Service Logic	Validation, trim, lower-case, time-stamping all in service classes
Database integration (Part B)	JPA entities, Spring Data repositories, H2 file-based DB
CRUD (Part B)	Create / Read / Update / Delete pages for articles
Interface	Polished login card, gradient theme, navigation bar, article cards, data table
Code Quality	Header on every file; inline comments on every annotation
Demo Explanation	Covered by this document

2. Architecture & Request Flow

Two flows operate side by side once Part B is added.

Flow 1 - Login (Part A, updated for sessions)

```
   [ Browser ]
       |  POST /login
       v
+--------------------+
|   AuthController   |   - reads username & password from the form
+--------------------+
       |  authService.isAuthenticated(form)
       v
+--------------------+
|    AuthService     |   - trims/lower-cases the username
+--------------------+
       |  adminRepository.findByUsername(...)
       v
+--------------------+
|  AdminRepository   |   - JPA interface; Spring writes the SQL
+--------------------+
       |  Optional<Admin>
       v
+--------------------+
|   AuthController   |   - sets session attribute "loggedInAdmin"
+--------------------+   - redirect:/admin
       v
   [ Browser ]   <- admin dashboard rendered by Thymeleaf
```

Flow 2 - Public wiki (Part B)

```
   [ Browser ]
       |  GET /wiki  (or /wiki/{id})
       v
+--------------------+
|   WikiController   |   - public; no login required
+--------------------+
       |
       v
+--------------------+
|   ArticleService   |   - findAll / findById / search / findByCategory
+--------------------+
       |
       v
+--------------------+
| ArticleRepository  |   - JPA interface
+--------------------+
       |  List<Article>  or  Optional<Article>
       v
   [ Browser ]   <- list page or single-article page
```

Flow 3 - Admin CRUD (Part B)

```
   [ Browser ]
       |  GET /admin/new                GET /admin/edit/{id}
       v                                 v
+--------------------+                +--------------------+
|  AdminController   |  -- session?  -> |  AdminController   |
+--------------------+   yes/no        +--------------------+
       |                                       |
       v                                       v
+--------------------+                +--------------------+
```

```
|   ArticleService   |                 |   ArticleService   |
+--------------------+                 +--------------------+
       |  save()                              |  save() (UPDATE because id is set)
       v                                      v
   redirect:/admin                        redirect:/admin

For DELETE: GET /admin/delete/{id} -> confirmation page
            POST /admin/delete/{id} -> articleService.deleteById(id) -> redirect:/admin
```

3. Project Structure

```
WikiApp/
+-- pom.xml                                  (Maven build)
+-- README.md
+-- .gitignore
+-- data/wikidb.mv.db                        (Part B - H2 file-based DB)
+-- src/main/
    +-- java/com/wikiapp/
    |   +-- WikiAppApplication.java          (entry point)
    |   +-- controller/
    |   |   +-- AuthController.java          (login / logout)
    |   |   +-- WikiController.java          (Part B - public)
    |   |   +-- AdminController.java         (Part B - protected CRUD)
    |   +-- service/
    |   |   +-- AuthService.java             (validates login against DB)
    |   |   +-- ArticleService.java          (Part B - article logic)
    |   +-- repository/
    |   |   +-- AdminRepository.java         (Part B - JPA)
    |   |   +-- ArticleRepository.java       (Part B - JPA)
    |   +-- model/
    |       +-- LoginForm.java               (form-backing POJO)
    |       +-- Admin.java                   (Part B - @Entity)
    |       +-- Article.java                 (Part B - @Entity)
    +-- resources/
        +-- application.properties           (port + DB config)
        +-- data.sql                         (Part B - seed admin + articles)
        +-- templates/
        |   +-- login.html
        |   +-- welcome.html / error.html / register.html
        |   +-- wiki/list.html               (Part B)
        |   +-- wiki/view.html               (Part B)
        |   +-- admin/dashboard.html         (Part B)
        |   +-- admin/form.html              (Part B - shared create/edit)
        |   +-- admin/confirm-delete.html    (Part B)
        +-- static/css/
            +-- style.css                    (login + Part B styles)
```

Each Java package matches one layer in the architecture. Templates are split into **wiki/** (public) and **admin/** (protected) subfolders to make the access pattern obvious at a glance.

PART A

Login screen with hard-coded credentials.

Note: AuthService and AuthController were upgraded in Part B to use the database and HttpSession. The walkthrough below describes the CURRENT, Part-B-aware versions of those files, so what you read matches the code that is actually running.

4. pom.xml - Maven Build File

File: WikiApp/pom.xml

```
<parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.4</version>
</parent>
```

What it does: Inherits Spring Boot's standard configuration so we do not have to pick versions for every dependency individually.

```
<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>

    <!-- PART B additions -->
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <scope>runtime</scope>
    </dependency>
</dependencies>
```

What it does: spring-boot-starter-web gives us @Controller, @GetMapping, @PostMapping and the embedded Tomcat server. **thymeleaf** turns HTML into dynamic pages. **validation** enables @NotBlank. **data-jpa (Part B)** gives us JpaRepository so we can talk to the database with simple interfaces. **h2 (Part B)** is a small SQL database that runs inside our app - no separate install needed.

5. WikiAppApplication.java - Entry Point

File: src/main/java/com/wikiapp/WikiAppApplication.java

```
@SpringBootApplication
public class WikiAppApplication {
    public static void main(String[] args) {
        SpringApplication.run(WikiAppApplication.class, args);
    }
}
```

What it does: @SpringBootApplication is a shortcut for three annotations: @Configuration (this class can define beans), @EnableAutoConfiguration (Spring sets up sensible defaults like the Tomcat server and now also the H2 database connection), and @ComponentScan (Spring searches this package and below for @Controller, @Service, @Repository and @Entity classes). The main method hands control to Spring Boot.

6. LoginForm.java - The Login Model

File: src/main/java/com/wikiapp/model/LoginForm.java

```
public class LoginForm {

    @NotBlank(message = "Username is required")
    private String username;

    @NotBlank(message = "Password is required")
    private String password;

    public LoginForm() { }

    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
}
```

What it does: A POJO (Plain Old Java Object) - private fields, a no-args constructor, getters and setters. Spring uses this class to receive the data the user typed into the login form. **@NotBlank** says the field must not be null and must not be empty/whitespace. The **private** fields plus get/set is **encapsulation**, a fundamental OOP principle.

7. AuthService.java - The Auth Service (Part B updated)

File: src/main/java/com/wikiapp/service/AuthService.java

```
@Service
public class AuthService {

    private final AdminRepository adminRepository;

    @Autowired
    public AuthService(AdminRepository adminRepository) {
        this.adminRepository = adminRepository;
    }
```

What it does: @Service marks this as a business-logic bean. **Constructor injection**: the Service declares it needs an AdminRepository, and Spring passes the bean it created earlier into the constructor. **final** means we never reassign this reference - safer code.

```
public boolean isAuthenticated(LoginForm form) {
    if (form == null) return false;

    String username = form.getUsername();
    String password = form.getPassword();

    if (username == null || username.trim().isEmpty()) return false;
    if (password == null || password.trim().isEmpty()) return false;

    String cleanedUsername = username.trim().toLowerCase();
```

What it does: Validation rules that belong in the Service layer. We reject null forms and empty/whitespace fields. Then we trim and lower-case the username so 'Movindu' or ' movindu ' still works. Note we do NOT lower-case the password - passwords are case-sensitive.

```
    Optional<Admin> maybeAdmin = adminRepository.findByUsername(cleanedUsername);
    if (maybeAdmin.isEmpty()) return false;

    Admin admin = maybeAdmin.get();
    return admin.getPassword().equals(password);
}
```

What it does: Looks up the admin by username. **Optional<Admin>** means 'either an Admin or nothing' - safer than returning null because the caller is forced to handle the missing case. If no admin exists, fail. Otherwise compare the stored password to the supplied one with .equals().

8. AuthController.java - The Auth Controller (Part B updated)

File: src/main/java/com/wikiapp/controller/AuthController.java

```
@Controller
@RequestMapping("/")
public class AuthController {

    public static final String SESSION_ADMIN_KEY = "loggedInAdmin";

    private final AuthService authService;

    @Autowired
    public AuthController(AuthService authService) {
        this.authService = authService;
    }
```

What it does: @Controller marks this as a web controller; methods return view names (HTML files). The **SESSION_ADMIN_KEY** constant is the key under which we store the logged-in username in the HttpSession. Using a constant prevents typos.

```
@GetMapping({"/", "/login"})
public String showLoginPage(Model model) {
    model.addAttribute("loginForm", new LoginForm());
    return "login";
}
```

What it does: Handles GET requests to / or /login. We put an empty LoginForm into the Model so the form has something to bind to. Returning "login" tells Spring to render **templates/login.html**.

```
@PostMapping("/login")
public String processLogin(@ModelAttribute("loginForm") LoginForm loginForm,
                           HttpSession session,
                           Model model) {
    boolean ok = authService.isAuthenticated(loginForm);
    if (ok) {
        String cleaned = loginForm.getUsername().trim().toLowerCase();
        session.setAttribute(SESSION_ADMIN_KEY, cleaned);
        return "redirect:/admin";
    }
    model.addAttribute("errorMessage",
            "Invalid username or password. Please try again.");
    return "error";
}
```

What it does: Handles the login form submission. **@ModelAttribute** tells Spring to build a LoginForm from the form fields. We ask the Service if the credentials are valid; if yes, we store the username in the **HttpSession** and **redirect** the browser to the admin dashboard. The 'redirect:' prefix tells Spring to send a 302 response instead of rendering a template - this avoids the browser's 'resubmit form?' warning if the user refreshes.

```
@GetMapping("/logout")
public String logout(HttpSession session) {
    session.invalidate();
    return "redirect:/login";
}
```

What it does: Handles logout. **session.invalidate()** throws away every value stored in this user's session, so the next request looks unauthenticated. Then we redirect back to the login page.

9. application.properties - Spring Boot Configuration

File: src/main/resources/application.properties

```
server.port=8080
spring.thymeleaf.cache=false
spring.application.name=WikiApp
```

What it does: Tomcat runs on port 8080. Thymeleaf caching is off so HTML edits show up without restarting. The application has a friendly log name.

```
# Part B - H2 file-based database
spring.datasource.url=jdbc:h2:file:./data/wikidb;AUTO_SERVER=TRUE
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

What it does: Configures the H2 database. The data is stored in files inside **./data/** so it survives restarts. The H2 console is enabled at **/h2-console** - useful for showing the actual database tables during the demo.

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always
```

What it does: ddl-auto=update tells Hibernate to create missing tables and add new columns automatically based on our @Entity classes. **defer-datasource-initialization** ensures Hibernate creates the tables BEFORE data.sql runs, so the seed inserts succeed.

10. login.html - The Login Form

File: src/main/resources/templates/login.html

```
<html lang="en" xmlns:th="http://www.thymeleaf.org">
```

What it does: The xmlns:th namespace unlocks the th:* attributes.

```
<form th:action="@{/login}" th:object="${loginForm}" method="post">
    <label for="username">Username</label>
    <input type="text" id="username" th:field="*{username}" autofocus/>

    <label for="password">Password</label>
    <input type="password" id="password" th:field="*{password}"/>

    <a class="btn secondary" th:href="@{/register}">Register</a>
    <button class="btn primary" type="submit">Log in</button>
</form>
```

What it does: The form posts to /login. **th:object** binds the whole form to the LoginForm object the controller put in the model. **th:field="*{username}"** automatically generates name=, id= and value= attributes that map to the LoginForm's username property. When submitted, Spring calls setUsername() with whatever was typed.

11. welcome.html / error.html / register.html

File: src/main/resources/templates/

```
<!-- welcome.html -->
<p>Hello <strong th:text="${username}">user</strong>, you have logged in successfully.</p>
<a class="btn primary" th:href="@{/login}">Log out</a>
```

welcome.html: Originally shown after a successful Part A login. With Part B added, the controller redirects straight to /admin instead, but this template stays as a backup view. **th:text** replaces the tag's body with the model attribute.

```
<!-- error.html -->
<p th:text="${errorMessage}">Something went wrong.</p>
<a class="btn primary" th:href="@{/login}">Try again</a>
```

error.html: Shown for failed logins and 'article not found' errors. The same template handles both cases - the controller decides what message to put in the model.

```
<!-- register.html -->
<p th:text="${message}">Coming soon.</p>
<a class="btn primary" th:href="@{/login}">Back to Login</a>
```

register.html: Placeholder. The brief did not require self-registration; admin accounts are seeded into the database at startup instead.

12. style.css - Page Styling

File: src/main/resources/static/css/style.css

```
body {
    background: linear-gradient(135deg, #1e3a8a 0%, #2563eb 50%, #38bdf8 100%);
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
}
```

Login page: Centred white card on a gradient background using flexbox. **100vh** means 100% of the viewport height.

```
/* Part B - top navigation bar */
.topbar {
    background: #1e3a8a;
    color: #ffffff;
    padding: 14px 28px;
    display: flex;
    justify-content: space-between;
    align-items: center;
}
.topbar.admin { background: #111827; }
```

Part B nav bar: Used on every wiki and admin page. The **.admin** class switches to a darker colour so it is visually obvious which area you are in.

```
/* Part B - article cards */
.article-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
    gap: 18px;
}
```

Article grid: CSS Grid creates a responsive layout - cards are at least 280px wide and the browser fits as many per row as possible. Resize the window to see it adapt.

```
/* Part B - admin data table */
.data-table { width: 100%; border-collapse: collapse; }
.data-table th { background: #f9fafb; }
.data-table tbody tr:hover { background: #f9fafb; }
```

Admin table: Standard table styling with a hover effect on each row to indicate it is interactive.

PART B

Database, public wiki, and admin CRUD.

13. Article.java - JPA Entity

File: src/main/java/com/wikiapp/model/Article.java

```
@Entity
public class Article {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

What it does: **@Entity** tells JPA to create a database table for this class. **@Id** marks the primary key (the unique row identifier). **@GeneratedValue(IDENTITY)** tells the database to auto-generate the id when we save a new article.

```
    @NotBlank(message = "Title is required")
    @Column(nullable = false)
    private String title;

    @NotBlank(message = "Category is required")
    @Column(nullable = false)
    private String category;

    @NotBlank(message = "Content is required")
    @Column(nullable = false, length = 5000)
    private String content;

    private LocalDateTime lastUpdated;
```

What it does: Three text fields plus a timestamp. **@NotBlank** validates the form input. **@Column(nullable = false)** makes the database column NOT NULL too. **length = 5000** overrides the default 255-char limit so we can store long article content. **LocalDateTime** is the modern Java date/time type.

```
    public Article() { }

    public Article(String title, String category, String content) {
        this.title = title;
        this.category = category;
        this.content = content;
        this.lastUpdated = LocalDateTime.now();
    }

    // ... getters and setters for all fields ...
```

What it does: A no-args constructor (required by JPA so it can build a blank object and fill it in from the database row), and a convenience constructor for creating articles in code. Standard getters and setters follow.

14. Admin.java - JPA Entity

File: src/main/java/com/wikiapp/model/Admin.java

```
@Entity
public class Admin {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @Column(nullable = false)
    private String password;

    public Admin() { }
    public Admin(String username, String password) {
        this.username = username;
        this.password = password;
    }
    // ... getters and setters ...
}
```

What it does: Replaces the hard-coded credentials we used in Part A. Each row in the ADMIN table is one administrator. **unique = true** on username means the database itself rejects duplicate usernames. The password is stored in plain text for simplicity (the brief does not require hashing). In a real system we would always hash passwords with BCrypt or Argon2.

15. ArticleRepository.java - JPA Interface

File: src/main/java/com/wikiapp/repository/ArticleRepository.java

```
public interface ArticleRepository extends JpaRepository<Article, Long> {

    List<Article> findByCategoryIgnoreCase(String category);

    List<Article> findByTitleContainingIgnoreCase(String text);
}
```

What it does: The most powerful single line in the project. By extending **JpaRepository<Article, Long>**, we get a fully working repository with no code: findAll(), findById(id), save(article), deleteById(id), count(), and many more. Spring writes all the SQL for us. The two custom methods follow Spring Data's **naming convention**: Spring reads the method name and writes a query for it. **findByCategoryIgnoreCase** becomes 'SELECT * FROM article WHERE LOWER(category) = LOWER(?)'. We never write SQL ourselves.

16. AdminRepository.java - JPA Interface

File: src/main/java/com/wikiapp/repository/AdminRepository.java

```
public interface AdminRepository extends JpaRepository<Admin, Long> {

    Optional<Admin> findByUsername(String username);
}
```

What it does: Same idea as ArticleRepository but for Admin. **findByUsername** is converted by Spring into the SQL 'SELECT * FROM admin WHERE username = ?'. Returning **Optional<Admin>** instead of Admin forces the caller to handle the case where no admin with that username exists - this prevents accidental NullPointerExceptions.

17. ArticleService.java - Business Logic

File: src/main/java/com/wikiapp/service/ArticleService.java

```
@Service
public class ArticleService {

    private final ArticleRepository articleRepository;

    @Autowired
    public ArticleService(ArticleRepository articleRepository) {
        this.articleRepository = articleRepository;
    }
```

What it does: Same Service-layer pattern we used in Part A. Constructor injection wires in the repository.

```
public List<Article> findAll() {
    return articleRepository.findAll();
}

public Optional<Article> findById(Long id) {
    return articleRepository.findById(id);
}

public List<Article> findByCategory(String category) {
    if (category == null || category.trim().isEmpty()) return findAll();
    return articleRepository.findByCategoryIgnoreCase(category.trim());
}

public List<Article> search(String text) {
    if (text == null || text.trim().isEmpty()) return findAll();
    return articleRepository.findByTitleContainingIgnoreCase(text.trim());
}
```

What it does: The READ side of CRUD. Each method is a thin wrapper around the repository, but with one small business rule: empty filter inputs fall back to listing all articles.

```
public Article save(Article article) {
    article.setLastUpdated(LocalDateTime.now());
    return articleRepository.save(article);
}

public void deleteById(Long id) {
    articleRepository.deleteById(id);
}

public boolean existsById(Long id) {
    return articleRepository.existsById(id);
}
```

What it does: CREATE/UPDATE/DELETE side. **save()** handles BOTH inserts (when article.id is null) and updates (when it has an id) - JPA decides automatically. We always stamp the lastUpdated field, which is a nice business rule that lives in the Service exactly where it should.

18. WikiController.java - Public Read Pages

File: src/main/java/com/wikiapp/controller/WikiController.java

```
@Controller
@RequestMapping("/wiki")
public class WikiController {

    private final ArticleService articleService;

    @Autowired
    public WikiController(ArticleService articleService) {
        this.articleService = articleService;
    }
```

What it does: @RequestMapping("/wiki") prefixes every method's URL with /wiki, so the listing is at /wiki and individual articles are at /wiki/{id}. These are the PUBLIC pages - no login is required.

```
@GetMapping
public String listArticles(
        @RequestParam(value = "category", required = false) String category,
        @RequestParam(value = "search",   required = false) String search,
        Model model) {

    List<Article> articles;
    if (search != null && !search.trim().isEmpty()) {
        articles = articleService.search(search);
    } else if (category != null && !category.trim().isEmpty()) {
        articles = articleService.findByCategory(category);
    } else {
        articles = articleService.findAll();
    }

    model.addAttribute("articles", articles);
    model.addAttribute("category", category);
    model.addAttribute("search", search);
    return "wiki/list";
}
```

What it does: Handles GET /wiki. **@RequestParam(required=false)** captures optional URL parameters - so /wiki?search=spring sets search='spring', and /wiki on its own makes both null. We pick the right Service method based on which (if any) parameter is present, then put the list and the current filter values into the Model so the template can show them.

```
@GetMapping("/{id}")
public String viewArticle(@PathVariable("id") Long id, Model model) {
    Optional<Article> maybeArticle = articleService.findById(id);
    if (maybeArticle.isEmpty()) {
        model.addAttribute("errorMessage", "The article you requested does not exist.");
        return "error";
    }
    model.addAttribute("article", maybeArticle.get());
    return "wiki/view";
}
```

What it does: Handles GET /wiki/{id} where {id} is a number. **@PathVariable** captures that value from the URL. If no article with that id exists, we send the user to a friendly error page instead of crashing.

19. AdminController.java - Protected CRUD

File: src/main/java/com/wikiapp/controller/AdminController.java

```
@Controller
@RequestMapping("/admin")
public class AdminController {

    private final ArticleService articleService;

    @Autowired
    public AdminController(ArticleService articleService) {
        this.articleService = articleService;
    }

    private boolean isLoggedIn(HttpSession session) {
        return session.getAttribute(AuthController.SESSION_ADMIN_KEY) != null;
    }
```

What it does: All admin URLs start with /admin. The **isLoggedIn** helper checks the session for the username we stored during login. Every admin endpoint calls this first - if it returns false, we redirect to /login. This is a very simple, viva-friendly authorisation check (no Spring Security needed).

```
@GetMapping
public String dashboard(HttpSession session, Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";

    model.addAttribute("articles", articleService.findAll());
    model.addAttribute("adminUser",
            session.getAttribute(AuthController.SESSION_ADMIN_KEY));
    return "admin/dashboard";
}
```

READ: Lists every article in a table for the admin to manage. We also pass the logged-in username so the navigation bar can greet them by name.

```
@GetMapping("/new")
public String showCreateForm(HttpSession session, Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";
    model.addAttribute("article", new Article());
    model.addAttribute("mode", "create");
    return "admin/form";
}

@PostMapping
public String createArticle(@Valid @ModelAttribute("article") Article article,
                            BindingResult bindingResult,
                            HttpSession session,
                            Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";
    if (bindingResult.hasErrors()) {
        model.addAttribute("mode", "create");
        return "admin/form";
    }
    articleService.save(article);
    return "redirect:/admin";
}
```

CREATE: Two methods - one shows the empty form (GET /admin/new), the other handles the submission (POST /admin). **@Valid** activates the @NotBlank checks on the Article. **BindingResult** collects any errors. If the input is invalid we redisplay the form with the error messages; otherwise we

save and redirect.

```
@GetMapping("/edit/{id}")
public String showEditForm(@PathVariable("id") Long id,
                           HttpSession session, Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";
    Optional<Article> maybeArticle = articleService.findById(id);
    if (maybeArticle.isEmpty()) {
        model.addAttribute("errorMessage", "The article you tried to edit does not exist.");
        return "error";
    }
    model.addAttribute("article", maybeArticle.get());
    model.addAttribute("mode", "edit");
    return "admin/form";
}

@PostMapping("/edit/{id}")
public String updateArticle(@PathVariable("id") Long id,
                            @Valid @ModelAttribute("article") Article article,
                            BindingResult bindingResult,
                            HttpSession session, Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";
    if (bindingResult.hasErrors()) {
        model.addAttribute("mode", "edit");
        return "admin/form";
    }
    article.setId(id);
    articleService.save(article);
    return "redirect:/admin";
}
```

UPDATE: Same two-step pattern. The GET pre-fills the form with the existing article; the POST saves changes. Setting **article.setId(id)** before save() is what makes JPA do an UPDATE rather than an INSERT - articles with an id get updated, articles without get inserted.

```
@GetMapping("/delete/{id}")
public String confirmDelete(@PathVariable("id") Long id,
                            HttpSession session, Model model) {
    if (!isLoggedIn(session)) return "redirect:/login";
    // ... show "are you sure?" page ...
    return "admin/confirm-delete";
}

@PostMapping("/delete/{id}")
public String deleteArticle(@PathVariable("id") Long id, HttpSession session) {
    if (!isLoggedIn(session)) return "redirect:/login";
    if (articleService.existsById(id)) articleService.deleteById(id);
    return "redirect:/admin";
}
```

DELETE: Two steps for safety. The GET shows a confirmation page; only the POST actually deletes. We use POST (not GET) for the delete because GET requests should never change data - this stops accidental deletions from URL prefetching, browser caching, or web crawlers.

20. data.sql - Database Seeding

File: src/main/resources/data.sql

```
MERGE INTO ADMIN (id, username, password) KEY(id)
    VALUES (1, 'movindu', '123');
```

What it does: Seeds the single admin account. **MERGE ... KEY(id)** means 'insert if a row with this id does not exist; otherwise update it' - so the script is safe to run repeatedly without crashing on duplicates.

```
MERGE INTO ARTICLE (id, title, category, content, last_updated) KEY(id) VALUES
    (1, 'Welcome to WikiApp', 'General', '...', CURRENT_TIMESTAMP);
-- ... and four more sample articles ...

ALTER TABLE ARTICLE ALTER COLUMN ID RESTART WITH 100;
ALTER TABLE ADMIN   ALTER COLUMN ID RESTART WITH 100;
```

What it does: Inserts five sample articles so the wiki has something to show on first run. **ALTER TABLE ... RESTART WITH 100** bumps the auto-increment counter past the seeded ids, so when the admin creates a new article it gets id 100, 101, 102 ... rather than colliding with the seeded ones.

21. wiki/list.html - Public Articles List

File: src/main/resources/templates/wiki/list.html

```
<form class="search" th:action="@{/wiki}" method="get">
    <input type="text" name="search" th:value="${search}"
           placeholder="Search articles by title..."/>
    <button class="btn primary" type="submit">Search</button>
    <a class="btn secondary" th:href="@{/wiki}">Clear</a>
</form>
```

Search bar: method="get" puts the search text into the URL (/wiki?search=spring) - this means the user can bookmark or share search results. **th:value** repopulates the box with the previous search so the user can see what they searched for.

```
<div class="article-grid" th:unless="${#lists.isEmpty(articles)}">
    <article class="article-card" th:each="a : ${articles}">
        <span class="badge" th:text="${a.category}">Category</span>
        <h2><a th:href="@{'/wiki/' + ${a.id}}" th:text="${a.title}">Title</a></h2>
        <p class="preview" th:text="${#strings.abbreviate(a.content, 140)}">Preview</p>
        <p class="meta">Last updated:
            <span th:text="${#temporals.format(a.lastUpdated, 'dd MMM yyyy HH:mm')}">date</span>
        </p>
    </article>
</div>
```

Article grid: th:each="a : ${articles}" is Thymeleaf's for-each loop - exactly like a Java for loop over a list. We render one card per article. **#strings.abbreviate(text, 140)** truncates the content to a 140-char preview. **#temporals.format(...)** formats the LocalDateTime into a friendly date string.

22. wiki/view.html - Single Article

File: src/main/resources/templates/wiki/view.html

```
<article class="article-full">
    <span class="badge" th:text="${article.category}">Category</span>
    <h1 th:text="${article.title}">Title</h1>
    <p class="meta">Last updated:
        <span th:text="${#temporals.format(article.lastUpdated, 'dd MMM yyyy HH:mm')}">date</span>
    </p>
    <div class="content" th:text="${article.content}">Content</div>
</article>
```

What it does: Renders one full article. **th:text** automatically HTML-escapes the content, so even if someone tries to put <script> tags into an article, they will appear as plain text - never executed. This is built-in XSS protection.

23. admin/dashboard.html - Admin Home

File: src/main/resources/templates/admin/dashboard.html

```
<nav class="topbar admin">
    <a class="brand" th:href="@{/admin}">WikiApp Admin</a>
    <div class="links">
        <span>Logged in as <strong th:text="${adminUser}">admin</strong></span>
        <a th:href="@{/wiki}">View public wiki</a>
        <a th:href="@{/logout}">Log out</a>
    </div>
</nav>
```

Top bar: Greets the logged-in admin by name (read from the session attribute) and provides links to the public wiki and the logout endpoint.

```
<table class="data-table" th:unless="${#lists.isEmpty(articles)}">
    <thead><tr>
        <th>ID</th><th>Title</th><th>Category</th>
        <th>Last updated</th><th>Actions</th>
    </tr></thead>
    <tbody>
        <tr th:each="a : ${articles}">
            <td th:text="${a.id}">1</td>
            <td><a th:href="@{'/wiki/' + ${a.id}}" th:text="${a.title}">Title</a></td>
            <td><span class="badge" th:text="${a.category}">cat</span></td>
            <td th:text="${#temporals.format(a.lastUpdated, 'dd MMM yyyy HH:mm')}">date</td>
            <td class="actions">
                <a class="btn small"        th:href="@{'/admin/edit/'   + ${a.id}}">Edit</a>
                <a class="btn small danger" th:href="@{'/admin/delete/' + ${a.id}}">Delete</a>
            </td>
        </tr>
    </tbody>
</table>
```

Article table: Loops over every article and renders a row with Edit and Delete buttons. The URLs use Thymeleaf string concatenation: **@{'/admin/edit/' + ${a.id}}** builds /admin/edit/100 for an article with id 100.

24. admin/form.html - Create / Edit Form

File: src/main/resources/templates/admin/form.html

```
<h1 th:text="${mode == 'edit'} ? 'Edit article' : 'New article'">Article form</h1>

<form th:action="${mode == 'edit'}
                 ? @{'/admin/edit/' + ${article.id}}
                 : @{/admin}"
      th:object="${article}" method="post" class="article-form">
```

What it does: ONE template handles both 'create' and 'edit'. The **mode** attribute (set by the controller) decides which heading and which form URL to use. The Thymeleaf ternary **${cond} ? a : b** is the same as Java's a?b:c.

```
<label for="title">Title</label>
<input type="text" id="title" th:field="*{title}"/>
<p class="error-text" th:if="${#fields.hasErrors('title')}" th:errors="*{title}"></p>

<label for="category">Category</label>
<input type="text" id="category" th:field="*{category}"/>
<p class="error-text" th:if="${#fields.hasErrors('category')}" th:errors="*{category}"></p>

<label for="content">Content</label>
<textarea id="content" th:field="*{content}" rows="10"></textarea>
<p class="error-text" th:if="${#fields.hasErrors('content')}" th:errors="*{content}"></p>
```

What it does: Three input fields bound to the Article object via **th:field**. **#fields.hasErrors('title')** checks if validation failed for this field, and **th:errors** displays the error message. This is how @NotBlank messages appear under the field that caused the error.

25. admin/confirm-delete.html - Delete Confirmation

File: src/main/resources/templates/admin/confirm-delete.html

```
<div class="card error">
    <h1>Delete article?</h1>
    <p>You are about to permanently delete this article:</p>
    <blockquote>
        <strong th:text="${article.title}">Title</strong><br/>
        <span class="badge" th:text="${article.category}">Category</span>
    </blockquote>

    <form th:action="@{'/admin/delete/' + ${article.id}}" method="post" class="buttons">
        <a class="btn secondary" th:href="@{/admin}">Cancel</a>
        <button class="btn danger" type="submit">Yes, delete</button>
    </form>
</div>
```

What it does: 'Are you sure?' page. The actual delete is a POST form (not just a link) so deleting requires an explicit click and cannot be triggered by URL prefetching, browser caching or accidental clicks. The Cancel button is a normal link back to the dashboard.

REFERENCE

26. Annotation Cheat Sheet

Every annotation used in the project, in plain English.

Spring core (Part A + B)

@SpringBootApplication	@Configuration + @EnableAutoConfiguration + @ComponentScan.
@Controller	Web controller; methods return view names (HTML files).
@Service	Business-logic bean. Spring manages it as a singleton.
@Repository	Data-access bean. Marks a class for data access.
@RequestMapping("/")	URL prefix at class level.
@GetMapping("/x")	Handle HTTP GET at /x. Used for viewing pages.
@PostMapping("/x")	Handle HTTP POST at /x. Used for form submissions.
@ModelAttribute	Build a Java object from form fields.
@Autowired	Inject the dependency. Used on the constructor.
@PathVariable	Capture a value from the URL, e.g. /wiki/{id}.
@RequestParam	Capture a query-string parameter, e.g. ?search=foo.
@Valid	Run validation rules on the bound object.
@NotBlank	Field must not be null or empty/whitespace.

JPA (Part B)

@Entity	This class maps to a database table.
@Id	This field is the primary key.
@GeneratedValue(IDENTITY)	Auto-generated id - the database picks the next number.
@Column(nullable=false)	The database column cannot hold NULL.
@Column(unique=true)	The database enforces uniqueness.
@Column(length=5000)	Override the default 255-char limit.

27. Thymeleaf Cheat Sheet

Every Thymeleaf attribute used in the templates.

xmlns:th="..."	Tells the parser to process th:* attributes.
@{/login}	URL expression - safer than hard-coding paths.
${variable}	Variable expression - reads from the Model.
***{property}**	Selection expression - reads from the th:object.
th:object="${form}"	Bind the whole form to this Model object.
th:field="*{name}"	Bind input to a property; sets name, id and value.
th:text="${x}"	Replace the tag's text with x. HTML-escaped automatically.
th:href="@{/x}"	Build a URL for an <a> tag's href.
th:action="@{/x}"	Build a URL for a form's submit target.
th:value="${x}"	Set an input's value attribute.
th:if="${cond}"	Render the element only when cond is true.
th:unless="${cond}"	Render the element only when cond is false.
th:each="a : ${list}"	For-each loop - render once per element.
th:errors="*{x}"	Show validation errors for the x property.
#fields.hasErrors('x')	True when the x property failed validation.
#strings.abbreviate(s,n)	Truncate string s to n characters.
#temporals.format(d,f)	Format a LocalDateTime using pattern f.
#lists.isEmpty(l)	True when the list is empty.

28. Likely Viva Questions & Answers

Read these out loud until they feel natural. The marker will probably ask 5-8 of them. Part B questions appear at the end.

Q: Walk me through what happens when I click 'Log in'.

A: The browser sends a POST request to /login with the username and password. Spring matches it to processLogin() in AuthController. Spring builds a LoginForm using its setters. The controller calls authService.isAuthenticated(loginForm). The service trims and lower-cases the username and asks adminRepository.findByUsername(). The repository returns an Optional<Admin>. If it is empty, fail. Otherwise we compare the stored password to the supplied one with .equals(). On success the controller stores the username in the HttpSession and returns 'redirect:/admin'. Spring sends a 302 response and the browser loads the admin dashboard.

Q: Why did you split your code into three layers?

A: Separation of concerns. The Controller handles HTTP. The Service holds the business rules. The Repository owns the data. When I added the database in Part B, only the Repository changed - the rest of the code stayed exactly the same. That is the reward of a layered design.

Q: What is the difference between @GetMapping and @PostMapping?

A: @GetMapping handles requests where the user just wants to view a page - it should not change anything. @PostMapping handles form submissions - it is used when the user is sending data, like a username and password or a new article.

Q: How does Spring know to inject AuthService into the Controller?

A: I marked AuthService with @Service so Spring creates one instance at startup. The Controller's constructor declares it needs an AuthService and I added @Autowired, so Spring passes the same instance in automatically. That is constructor dependency injection.

Q: How does the username typed in the form end up in your Java code?

A: The HTML input has th:field="*{username}". When the form is submitted, Spring sees @ModelAttribute LoginForm in the controller, creates a new LoginForm, calls setUsername() with the typed value, then passes the filled object into my method.

Q: What does Model do?

A: Model is a container for data going to the view. I put values in with addAttribute(). Thymeleaf reads from it when rendering the HTML, so ${username} prints whatever I stored under the name 'username'.

Q: What is a POJO?

A: Plain Old Java Object - a simple class with private fields, a no-args constructor, and getters and setters. LoginForm is a POJO.

Q: What does @SpringBootApplication actually do?

A: It is a shortcut for @Configuration (this class can define beans), @EnableAutoConfiguration (Spring sets up sensible defaults like the embedded Tomcat server and now the H2 database connection), and @ComponentScan (Spring finds my @Controller, @Service, @Repository and

@Entity classes).

Q: What is the embedded Tomcat?

A: Tomcat is the web server that listens on port 8080. Spring Boot includes it inside the application, so I do not need to install or configure a separate server - running my main method starts the server.

Q: What is the difference between == and .equals() for strings?

A: == compares whether two variables point to the same memory location. .equals() compares the actual character contents. For strings you should always use .equals().

Q: What is a bean?

A: An object that Spring manages. I do not create it with 'new' - Spring creates it once at startup, keeps a reference, and gives it to anyone that asks.

Q: What is encapsulation?

A: An OOP principle: hide internal state behind methods. My LoginForm has private fields and exposes them only through getters and setters.

Q: What is dependency injection?

A: Instead of a class creating its own dependencies with 'new', the dependencies are passed in from outside. Spring does this for me by matching constructor parameters to beans.

Q: Walk me through what happens when an admin creates a new article.

A: The admin clicks 'New article' on the dashboard, which sends GET /admin/new. AdminController checks the session, then puts an empty Article into the model and renders admin/form.html. The form is bound to that Article via th:object. When the admin submits, POST /admin runs createArticle(). @ModelAttribute fills in an Article, @Valid runs the @NotBlank checks, BindingResult collects any errors. If there are no errors, articleService.save(article) stamps lastUpdated and asks the JPA repository to INSERT a new row. The controller then returns redirect:/admin so the user lands back on the dashboard with their new article visible.

Q: How does JPA know what tables to create?

A: Hibernate (the JPA implementation Spring Boot ships with) scans for @Entity classes at startup. For each one, it creates a table with the same name and a column for each field. With spring.jpa.hibernate.ddl-auto=update it creates tables that are missing and adds new columns when I add new fields, but never drops data.

Q: What is JpaRepository and why is it just an interface?

A: JpaRepository is a Spring Data interface that declares the standard CRUD methods - findAll, findById, save, deleteById and many more. When I extend it, Spring scans my interface at startup and writes a class that implements every method, including the SQL. So I get a fully working data access layer with literally one line of code.

Q: How does Spring know what SQL to generate for findByCategoryIgnoreCase?

A: Spring Data parses the method name. 'findBy' means SELECT, 'Category' means WHERE category = ?, and 'IgnoreCase' wraps both sides in LOWER(). So the method becomes SELECT * FROM article WHERE LOWER(category) = LOWER(?). I never wrote a single line of SQL.

Q: What is @Entity?

A: An annotation from JPA that tells Hibernate to map this class to a database table. Each instance of the class is one row. The fields become columns - unless we mark one with @Transient.

Q: What is @Id and @GeneratedValue?

A: @Id marks a field as the primary key - the unique row identifier. @GeneratedValue(IDENTITY) tells the database to auto-generate the value, so I do not have to pick ids myself when creating new articles.

Q: Why use Optional<Admin> instead of Admin in findByUsername?

A: Because the lookup might find nothing. If I returned Admin and the username did not exist, I would have to return null - and the caller could forget to check, leading to a NullPointerException. Optional forces the caller to handle the empty case explicitly with isEmpty() or isPresent().

Q: How does HttpSession work?

A: When a browser first visits the site, Tomcat assigns it a unique session id and sends it back as a cookie. On every later request the browser sends that cookie, so Tomcat knows which session to load. session.setAttribute(key, value) stores something in memory tied to that session. session.invalidate() throws everything away - that is how logout works.

Q: Why redirect after login instead of rendering the dashboard directly?

A: Two reasons. One: it changes the URL in the address bar from /login to /admin, so the user can bookmark or refresh without resubmitting the form. Two: most browsers warn 'do you want to resubmit?' on refreshes of POST results - redirect avoids that warning.

Q: How is the admin area protected?

A: Every method in AdminController calls isLoggedIn(session) first. That helper just checks whether session.getAttribute("loggedInAdmin") is null. If it is, the method returns 'redirect:/login' so the user is sent to the login page. It is a simple, explicit check that anyone can read and understand.

Q: Why use POST for delete and not GET?

A: GET requests are supposed to be safe - they should never change data. Browsers prefetch GET URLs, web crawlers follow them, and they get cached. If delete were a GET, any of those could accidentally wipe an article. Using POST means the browser will only fire it when the user explicitly submits the form.

Q: Why are articles stored in plain text but in a real app you would not store passwords in plain text?

A: Article content is meant to be read - storing it as plain text is correct. Passwords are secrets - if the database leaks, plain-text passwords let an attacker log in as anyone. In a real app I would hash passwords with BCrypt before storing. The brief did not require it, so I kept the code simple to explain.

Q: What does redirect: do?

A: It tells Spring to send a 302 HTTP response with a Location header pointing to a different URL, instead of rendering a template. The browser then makes a fresh GET request to that URL.

Q: How do you stop someone from forging a session cookie?

A: Tomcat generates session ids that are long random strings, so they cannot be guessed. In a production deployment we would also set the cookie to HttpOnly (not readable by JavaScript) and Secure (only sent over HTTPS). For Part B that level of hardening was not in scope.

Q: Why use one form template for both create and edit?

A: DRY - Don't Repeat Yourself. The form looks the same; only the title and the submit URL differ. I pass a 'mode' attribute to decide between them with a Thymeleaf ternary. If I needed to change the form layout, I only change one file.

Q: What does .save() actually do for a new vs existing entity?

A: If the entity's id is null, JPA does an INSERT and the database fills in the id. If the id is already set, JPA does an UPDATE on the row with that id. That is why my updateArticle() controller does article.setId(id) before calling save() - it tells JPA which row to update.

Q: What would be the first thing you would change about this project if you had more time?

A: Hash passwords with BCrypt and add proper Spring Security so I do not have to write the session-check helper myself. Then maybe add image uploads and a markdown editor for the article content.

29. One-Minute Elevator Pitch

If asked 'tell me about your project', say this. Practice it out loud until it flows naturally.

"WikiApp is a Spring Boot project I built in two stages for BIT235. Part A was a simple login screen using the MVC pattern with three layers: Controller, Service and Repository. Part B replaced the hard-coded credentials with a JPA-managed database using H2, and added a public wiki for browsing articles plus an admin back-end with full CRUD - create, read, update and delete. The login now stores the admin's username in an HttpSession, and every admin endpoint checks that session before doing anything. The clean separation of layers meant the upgrade from Part A to Part B only required changing the Repository - the Controller and Service barely moved. The code uses fundamental Java only: classes, getters and setters, Spring annotations, simple if-statements and the JpaRepository interface. No streams, no lambdas, no Spring Security - everything I built I can explain."

30. Vocabulary Glossary

MVC	Model-View-Controller pattern. Model = data, View = HTML, Controller = traffic cop.
POJO	Plain Old Java Object. Private fields + no-args constructor + getters/setters.
Bean	An object that Spring creates and manages on your behalf.
Dependency Injection (DI)	Spring passes required objects in instead of the class creating them.
Constructor Injection	DI through constructor parameters - the technique I used.
HTTP GET	A request asking the server for a page. Should not change anything.
HTTP POST	A request submitting data to the server. Used by forms.
HTTP 302 redirect	Server says 'go look at this other URL instead'.
Thymeleaf	The template engine that turns my .html files into dynamic web pages.
Embedded Tomcat	The web server bundled inside the application.
Hard-coded	A value written directly in the source code, not loaded from a database.
Encapsulation	OOP principle: hide internal state behind getters and setters.
Annotation	Metadata starting with @, e.g. @Controller. Tells Spring how to treat the class.
Layered architecture	Splitting code into Controller / Service / Repository layers.
Maven	The build tool. Reads pom.xml, downloads dependencies, compiles, packages, runs the app.
Spring Boot	A framework that makes Spring apps easy to set up and run.
JPA	Java Persistence API - the standard way to map Java objects to database tables.
Hibernate	The JPA implementation that Spring Boot ships with.
Entity	A Java class marked @Entity that maps to a database table.
Repository	A class or interface that talks to the database. JpaRepository in Part B.
CRUD	Create, Read, Update, Delete - the four basic database operations.

HttpSession	Server-side storage tied to a single user's browser via a cookie.
H2 database	A small SQL database that runs inside the application. No install needed.
Optional	A wrapper that means 'either a T or nothing' - safer than returning null.
Foreign key (FK)	A column that references the primary key of another table.
Primary key (PK)	The column that uniquely identifies each row in a table.

End of report - good luck with the viva!